

## WEEK 2: PRODUCT AND CATEGORY MANAGEMENT SYSTEM

DEEPIKAA S

### Scenario

The e-commerce company wants to maintain information about different products and their categories. Each product belongs to a particular category. The system should store and manage details such as Product Name, Price, and Stock Availability.

### 1. Design Product and Category Tables

#### Category Table

The Category table is used to store information about different product categories. The Category\_ID uniquely identifies each category.

```
SQL> CREATE TABLE Category1(Category_ID VARCHAR2(7),Category_Name VARCHAR2(20),Description VARCHAR2(50));  
Table created.
```

#### Product Table

The Product table is used to store details about the products. It contains information such as Product\_ID, Product\_Name, Price, Stock, and Category\_ID.

```
SQL> CREATE TABLE Product1(Product_ID VARCHAR2(7),Product_Name VARCHAR2(20),Category_ID VARCHAR2(7),Price NUMBER(9,2),Stock_Quantity  
Number);  
Table created.
```

The two tables, Category and Product, have been created. Now, we need to define the relationship between the tables and apply the required constraints.

### 2. Define Relationships and Constraints

#### Relationship Between Category and Product

A category can contain many products, but each product belongs to only one category.

Therefore, the relationship between the two tables is:

#### One Category → Many Products

#### Relationship: One-to-Many (1 : )

#### Primary Key

A Primary Key is a column that uniquely identifies each record in a table. It does not allow duplicate values.

- Category\_ID is the Primary Key in the Category table.
- Product\_ID is the Primary Key in the Product table.

```
SQL> ALTER TABLE Category ADD CONSTRAINT PK_category PRIMARY KEY(category_ID);  
Table altered.
```

```
SQL> ALTER TABLE Product ADD CONSTRAINT PK_product PRIMARY KEY(product_ID);  
Table altered.
```

### Foreign Key

A Foreign Key is used to connect two tables.

The Category\_ID in the Product table acts as the Foreign Key. It refers to the Category\_ID in the Category table.

```
SQL> ALTER TABLE Product1 ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category1(Category_ID);  
Table altered.
```

### NOT NULL Constraint

The NOT NULL constraint is used when a column must contain a value. It does not allow the field to be left empty.

```
SQL> ALTER TABLE Category1 MODIFY Category_Name VARCHAR2(100) NOT NULL;  
Table altered.  
  
SQL> ALTER TABLE Product1 MODIFY Product_Name VARCHAR2(100) NOT NULL;  
Table altered.
```

### CHECK Constraint

The CHECK constraint is used to make sure that the value entered satisfies a particular condition.

For example, the product price can be checked to make sure that it is greater than zero.

```
SQL> ALTER TABLE Product1 ADD CONSTRAINT CHK_Price CHECK (PRICE > 0);  
Table altered.  
  
SQL> ALTER TABLE Product1 ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);  
Table altered.
```

### UNIQUE Constraint

The UNIQUE constraint is used to make sure that duplicate values are not entered into a column.

```
SQL> ALTER TABLE Category1 ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);  
Table altered.
```

## 3. Insert Values

### Category Table

Values are inserted into the Category table using the INSERT command.

```

SQL> INSERT INTO Category1 VALUES('&Category_ID','&Category_Name','&Description');
Enter value for category_id: CAT101
Enter value for category_name: Electronics
Enter value for description: Electronic Gadgets and Devices
old 1: INSERT INTO Category1 VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category1 VALUES(' CAT101',' Electronics','Electronic Gadgets and Devices')

1 row created.

SQL> /
Enter value for category_id: CAT102
Enter value for category_name: Clothing
Enter value for description: Men,Women and kids apparels
old 1: INSERT INTO Category1 VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category1 VALUES(' CAT102',' Clothing','Men,Women and kids apparels')

1 row created.

SQL> /
Enter value for category_id: CAT103
Enter value for category_name: Grocery
Enter value for description: Food items and daily essentials
old 1: INSERT INTO Category1 VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category1 VALUES('CAT103',' Grocery','Food items and daily essentials')
INSERT INTO Category1 VALUES('CAT103',' Grocery','Food items and daily essentials')

```

```

SQL> /
Enter value for category_id: CAT104
Enter value for category_name: Furniture
Enter value for description: Home and office Furniture
old 1: INSERT INTO Category1 VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category1 VALUES('CAT104','Furniture',' Home and office Furniture')

1 row created.

SQL> /
Enter value for category_id: CAT105
Enter value for category_name: Books
Enter value for description: Educational and general Books
old 1: INSERT INTO Category1 VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category1 VALUES('CAT105','Books','Educational and general Books')

1 row created.

```

## Product Table

Values are inserted into the Product table using the INSERT command.

```

SQL> INSERT INTO Product1 VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity);
Enter value for product_id: P101
Enter value for product_name: Shoe
Enter value for category_id: CAT101
Enter value for price: 378
Enter value for stock_quantity: 40
old 1: INSERT INTO Product1 VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product1 VALUES('P101','Shoe','CAT101',378,40)

1 row created.

SQL> /
Enter value for product_id: P111
Enter value for product_name: Sketch
Enter value for category_id: CAT101
Enter value for price: 28
Enter value for stock_quantity: 10
old 1: INSERT INTO Product1 VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product1 VALUES('P111','Sketch','CAT101',28,10)

1 row created.

```

Like this, insert a few more rows.

```

SQL> CONNECT
Enter user-name: C##Deepikaa
Enter password:
Connected.
SQL> /
SP2-0103: Nothing in SQL buffer to run.
SQL> INSERT INTO Product1 VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity);
Enter value for product_id: P105
Enter value for product_name: Laptop Bag
Enter value for category_id: CAT102
Enter value for price: 799
Enter value for stock_quantity: 12
old 1: INSERT INTO Product1 VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product1 VALUES('P105','Laptop Bag','CAT102',799,12)

1 row created.

SQL> P106
SP2-0042: unknown command "P106" - rest of line ignored.
SQL> /
Enter value for product_id: P106
Enter value for product_name: Notebook
Enter value for category_id: CAT105
Enter value for price: 120
Enter value for stock_quantity: 30
old 1: INSERT INTO Product1 VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product1 VALUES('P106','Notebook','CAT105',120,30)

1 row created.

```

#### After Insertion – Category Table

```

SQL> INSERT INTO PRODUCT1 VALUES ('P121','Cot','CAT103',19000,7);

1 row created.

```

#### 4. CRUD Operations

CRUD stands for:

- C – Create
- R – Read
- U – Update
- D – Delete

CRUD operations are used to manage the records stored in the database.

##### Insert Record

A new product can be added to the Product table using the INSERT command.

```

SQL> INSERT INTO PRODUCT1 VALUES ('P121','Cot','CAT103',19000,7);

1 row created.

```

##### Update Price

The price of an existing product can be changed using the UPDATE command.

```
SQL> UPDATE PRODUCT1 SET Price = 56000 WHERE Product_Name = 'Cot';

1 row updated.
```

## Delete a Product

An existing product can be removed from the Product table using the DELETE command.

```
SQL> DELETE FROM PRODUCT1 WHERE Product_ID = 'P121';

1 row deleted.
```

## 5. Generate Category-wise Product Reports

### 1. Display all products under each category

This report is used to display all the products belonging to each category.

```
SQL> SELECT C.CATEGORY_ID,C.CATEGORY_NAME,P.PRODUCT_ID,P.PRICE,P.STOCK_QUANTITY FROM CATEGORY1 C JOIN PRODUCT1 P ON C.CATEGORY_ID = P
.CATEGORY_ID ORDER BY C.CATEGORY_ID;
```

| CATEGORY_ID | CATEGORY_NAME | PRODUCT_ID | PRICE | STOCK_QUANTITY |
|-------------|---------------|------------|-------|----------------|
| CAT101      | Electronics   | P107       | 56000 | 45             |
| CAT101      | Electronics   | P111       | 28    | 10             |
| CAT101      | Electronics   | P101       | 378   | 40             |

### 2. Count the number of products in each category

This report shows how many products are available under each category.

```
SQL> SELECT C.CATEGORY_NAME, COUNT(P.PRODUCT_ID) AS Product_Count FROM PRODUCT1 P JOIN CATEGORY1 C ON P.CATEGORY_ID = C.CATEGORY_ID G
ROUP BY C.CATEGORY_NAME;
```

| CATEGORY_NAME | PRODUCT_COUNT |
|---------------|---------------|
| Clothing      | 2             |
| Books         | 1             |
| Electronics   | 4             |

### 3. Find the highest-priced product in each category

This report is used to find the product with the highest price in every category.

```
SQL> SELECT C.CATEGORY_NAME, P.PRODUCT_NAME, P.PRICE AS HIGHEST_PRICE FROM PRODUCT1 P JOIN CATEGORY1 C ON P.CATEGORY_ID = C.CATEGORY_ID WHERE P.PRICE = (SELECT MAX(P2.PRICE) FROM PRODUCT1 P2 WHERE P2.CATEGORY_ID = P.CATEGORY_ID);
```

```
CATEGORY_NAME
-----
PRODUCT_NAME
-----
HIGHEST_PRICE
-----
Clothing
Laptop Bag      799

Books
Notebook        120

CATEGORY_NAME
-----
PRODUCT_NAME
-----
HIGHEST_PRICE
-----

Electronics
PHONE        56000

Electronics
PHONE

CATEGORY_NAME
-----
PRODUCT_NAME
-----
```

#### 4. Display categories having more than 5 products

This report displays only those categories that contain more than five products.

```
SQL> SELECT C.CATEGORY_NAME FROM PRODUCT1 P JOIN CATEGORY1 C ON P.CATEGORY_ID = C.CATEGORY_ID GROUP BY C.CATEGORY_NAME HAVING COUNT(P.PRODUCT_ID) > 5;
```

```
no rows selected
```

#### 5. Find the average product price category-wise

This report calculates the average price of products available in each category.

```
SQL> SELECT C.CATEGORY_NAME, AVG(P.PRICE) AS AVG_PRICE FROM PRODUCT1 P JOIN CATEGORY1 C ON P.CATEGORY_ID = C.CATEGORY_ID GROUP BY C.CATEGORY_NAME;
```

```
CATEGORY_NAME
-----
AVG_PRICE
-----
Clothing
699

Books
120

Electronics
28101.5
```